Mesh Refinement and Multigrid Self-Gravity in the AthenaK Shearing Box

Implementation Document
Phase 0 (refinement infrastructure), Phase 1 (shear-periodic multigrid boundaries), Phase 2 (gravity on refined meshes)

`athenak-multigrid` tree, branch `multigrid`
commit range `cc4404ba..fef5c6c9` (plus the Phase-0a predecessors `e7ef37f2`, `cc4404ba`)

August 18, 2026

Abstract

This document describes *how* mesh refinement and the multigrid Poisson solver were made to work in the AthenaK shearing box — the data structures, algorithms, call sites, invariants, and guards — as a companion to `multigrid_selfgravity_validation.pdf`, which records *what the resulting code does* on test problems. Nothing here is a result; everything here is a mechanism.

The work divides into three phases. **Phase 0** lifts AthenaK's blanket prohibition on shearing box + refinement, replacing it with an enforced refinement *policy* (uniform level on both shear-periodic x_1 faces; complete azimuthal rings when orbital advection is active), makes orbital advection work level by level, adds a fully consistent non-FARGO (full-velocity) mode, and rebuilds all shear-boundary communication state after every adaptive mesh update. **Phase 1** makes the multigrid solver shear-aware by treating shear-periodicity as what it is — a boundary *identification* with a time-dependent azimuthal offset — so the 7-point Laplacian, smoother, and transfer operators are untouched and only the x_1 ghost fill changes: a gather/remap/scatter through per-level global boundary planes, using the same conservative remap-flux kernels as orbital advection. **Phase 2** carries that fill onto refined meshes: interior rings (2a) need only a guard relaxation, refined *boundary* annuli (2b) require the same algorithm re-expressed at multigrid-octet granularity in host code, and adaptive refinement (2c) requires every shear structure sized at construction to become rebuildable, which it does by riding the solver's existing remesh-detection path.

A final section documents the rank-consistency fix (`ShearingBox::ConsistentDx2`) that removed a deterministic MPI message-size desynchronization in the shear exchange, found at 64 ranks.

Contents

1 Scope, sources, and how to read this

1.1 What this document is

This is an implementation reference. For each phase it answers: what invariant does the code rely on, where is that invariant established and enforced, what data structure carries the new state, which function performs the work, and from which call site is it reached. Numerical results, convergence rates, and comparisons against the FFT solver live in the companion validation document and notebook:

`validation/multigrid_selfgravity_validation.pdf`	results, tests, figures
`validation/multigrid_validation_tests.ipynb`	the analysis behind them
`validation/fft_selfgravity_validation.pdf`	the FFT solver used as oracle

1.2 Commit provenance

The implementation commits, in order, are listed in Table 1. Commits that only touch documentation, notebooks, figures, or input files are omitted.

Commit	Phase	Content
`e7ef37f2`	0a	Refinement policy, per-level FARGO, non-FARGO mode
`cc4404ba`	0b	Uniformly refined x_1 boundary faces
`f0acd383`	0c	Derefinement-ordering fix (whole-list sort)
`b5388c02`	0c	AMR: rebuild x_1-boundary state per mesh update; graded cap
`e853a004`	1	`multigrid_shear.cpp`; shear-periodic MG boundaries
`be32edce`	1	Multi-rank shear fill (one `Allreduce` seam)
`d9eacaa2`	—	Tomida & Stone (2023) consistency tests; MG driver setters
`0cd084d2`	2a	Gravity on interior-ring refined meshes (guard relaxation)
`a214fddf`	2b	Sheared octet ghosts; scalar remap-flux twins
`008cb5a2`	2c	AMR + multigrid gravity (rebuild on remesh)
`89904070`	—	Rank-desync fix: `ConsistentDx2`, sender and unpack
`ca63f727`	—	Same fix, recv-posting sites (completes all seven)

Table 1: Implementation commits covered by this document.

1.3 Files touched

File	Role
New	
`src/multigrid/multigrid_shear.cpp`	The entire shear-periodic multigrid boundary implementation (458 lines): `ComputeShearQomt`, `ExecuteMGTaskList`, `AllocateShearPlanes`, `FillShearGhostPack`, `FillShearOctetGhosts`, `RootShearBoundaryX1`
Modified — mesh / refinement	
`src/mesh/mesh.cpp`	Prohibition replaced by policy warning
`src/mesh/build_tree.cpp`	`Mesh::CheckShearingBoxRefinement`
`src/mesh/mesh.hpp`	Declaration
`src/mesh/mesh_refinement.cpp`	Boundary veto, graded cap, ring-wise flag sync, post-update policy check, shear reinit hook, derefinement sort fix
`src/mesh/mesh_refinement.hpp`	`shearing_box_`, `sbox_ring_policy_`
Modified — shearing box	
`src/shearing_box/shearing_box.{cpp,hpp}`	`SetX1BndryMBs`, `ReinitAfterMeshUpdate`, `ConsistentDx2`, `orbital_advection` flag
`src/shearing_box/shearing_box_cc.cpp`	`AllocateBuffers`, non-FARGO azimuthal offsets, consistent `joffset`
`src/shearing_box/shearing_box_fc.cpp`	`AllocateBuffers`, consistent `joffset`
`src/shearing_box/shearing_box_tasks.cpp`	Consistent `joffset` at the three recv/clear sites
`src/shearing_box/shearing_box_srcterms.cpp`	Full-velocity source terms
`src/shearing_box/orbital_advection.cpp`	`maxjshift` estimate
`src/shearing_box/orbital_advection_cc.cpp`	Per-step shift guard
`src/shearing_box/remap_fluxes.hpp`	Scalar per-face remap twins
Modified — gravity / multigrid	
`src/gravity/mg_gravity.cpp`	Shear enablement, guards, remap-order parameter, mean-source subtraction, phase freeze
`src/multigrid/multigrid.hpp`	Shear members and declarations
`src/multigrid/multigrid.cpp`	Plane deallocation
`src/multigrid/multigrid_driver.cpp`	BC plumbing, octet wrap, octet face flags, octet fill hooks, root fill hook, AMR rebuild
`src/multigrid/multigrid_tasks.cpp`	Block-level fill hook in `PhysicalBoundary`
Modified — physics modules and pgens	
`src/hydro/hydro.cpp`, `src/mhd/mhd.cpp`	Optional orbital advection; MHD+FARGO+refinement fatal
`src/pgen/tests/shwave.cpp`, `swing.cpp`	Background shear in ICs; `MovingRingRefine` user AMR criterion
`src/CMakeLists.txt`	`multigrid/multigrid_shear.cpp`

2 Background: what the shearing box already had

The local shearing-box approximation solves the fluid equations in a frame corotating at Ω_0 at radius R_0, with background flow $\mathbf{v}_{\rm bg} = -q\Omega_0 x\,\hat{\mathbf{e}}_y$. The x_1 boundaries are *shear-periodic*: the domain is identified with its images displaced azimuthally at the shear rate,

$$u(x + L_x,\, y,\, z,\, t) \;=\; u(x,\; y + q\Omega_0 L_x t,\; z,\; t)\,, \tag{1}$$

which is a *boundary identification*, not a change of the interior equations. This single observation is the organizing idea of Phases 1 and 2.

AthenaK ships three pieces of pre-existing machinery that this work builds on.

The shear-periodic boundary exchange (`shearing_box_cc.cpp`, `shearing_box_fc.cpp`). `ShearingBoxCC` and `ShearingBoxFC` derive from `ShearingBox`. Each holds a list of the MeshBlocks on this rank that touch the $\pm x_1$ faces (`x1bndry_mbgid`), send/recv buffers, and MPI request arrays. The exchange sends each boundary block's ghost data to up to three azimuthal targets, chosen from the integer part of the shift, and applies the fractional part with a conservative remap.

Orbital advection / FARGO (`orbital_advection*.cpp`). In the default (shear-subtracted) frame, velocities are perturbations about $\mathbf{v}_{\rm bg}$ and the background advection is applied analytically: an integer cell shift by communication with x_2-face neighbors, plus a conservative fractional remap. The communicated buffer depth is `ng` + `maxjshift`.

Conservative remap fluxes (`remap_fluxes.hpp`). `DC_RemapFlx`, `PLM_RemapFlx`, `PPMX_RemapFlx` are Kokkos *team* kernels that, given a padded scratch row q and a fractional shift $\epsilon \in (-1, 1)$, write the remap flux at each cell face. A row is shifted by

$$q_j^{\rm new} \;=\; q_j - (F_{j+1} - F_j)\,, \tag{2}$$

which is conservative by construction. The same kernels serve orbital advection, the shear-periodic exchange, the FFT gravity solver, and (Phase 1) the multigrid solver; one shared implementation means the shift used by gravity and the shift used by the hydro solver cannot silently diverge.

The multigrid solver (`src/multigrid/`). An FAS multigrid Poisson solver, ported from Athena++ (Tomida & Stone 2023). Its hierarchy has three regimes, which matters greatly for Phase 2:

1. **Block levels**: one `Multigrid` object over the `MeshBlockPack`, coarsened down to one cell per block. Device data, per-block, communicated with boundary buffers.
2. **Octet levels**: below the block levels, on refined meshes, the mesh is represented as 2^3 *octets* per level. These live in *host* memory and are *replicated on every rank* (via an `Allgatherv` in the block→root transfer).
3. **Root grid**: a single global array, also replicated per rank, coarsened to the coarsest level and solved there.

On uniform grids there are no octet levels. Phase 1 therefore touches regimes 1 and 3; Phase 2b exists precisely because boundary refinement creates regime-2 objects sitting on the shear faces.

3 Phase 0: mesh refinement in the shearing box

Before this work, `mesh.cpp` contained an unconditional fatal:

```
// FIXME: The shearing box is not currently compatible with SMR/AMR
if (multilevel && pin->DoesBlockExist("shearing_box")) { ... std::exit(EXIT_FAILURE); }
```

Phase 0 replaces the prohibition with a *policy*: refinement configurations that preserve the assumptions of the shear machinery are allowed, everything else aborts with a message naming the violation. The policy is not a convenience; each clause corresponds to a specific assumption in existing code.

3.1 The two invariants, and what they protect

Invariant I1 (uniform boundary level). All MeshBlocks touching either shear-periodic x_1 face share *one* refinement level. That level may exceed the root level, provided both faces are refined uniformly over their full x_2/x_3 extent.

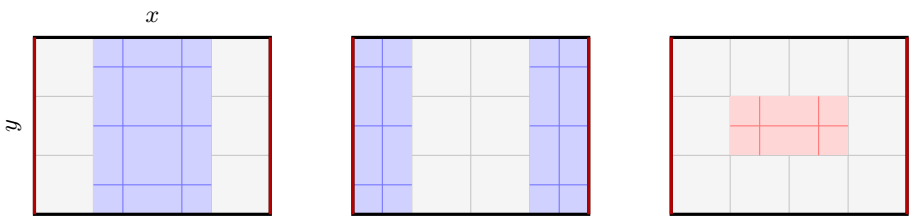
What it protects. A code audit (commit `cc4404ba`) established that nothing in the shear machinery references the root grid as such. The unshifted wrap across x_1 goes through the ordinary neighbor machinery — the mesh tree treats `shear_periodic` exactly like `periodic`, including 2:1 nesting — and the shear pass redistributes ghost data azimuthally *within* each face using per-block Δx_2 arithmetic and a level-aware `FindTargetMB`. Its real assumptions are only: (i) same-size blocks along each face, and (ii) a same-level wrap between the two faces. Refining both faces uniformly satisfies both, and is also physically natural — a structure at the shear boundary has images on *both* faces. Mixed-level boundaries remain fatal because combining the y-shift with prolongation inside the shear communication is not implemented.

Invariant I2 (annular rings, FARGO only). With `orbital_advection = true`, every refined region must span the full x_2 extent: for each (ℓ, l_{x1}, l_{x3}) the number of leaf blocks must equal $N^{\mathrm{root}}_{\mathrm{mb},x2} \ll (\ell - \ell_{\mathrm{root}})$.

What it protects. Orbital advection communicates only with x_2-face neighbors and applies a per-block integer shift. If a refined patch ended partway around the azimuth, a fine block would have a coarse x_2-face neighbor, and the per-block shift kernel plus the existing (same-size) communication pattern would both be invalid. Requiring complete rings makes every x_2-face neighbor same-level and same-size, so the existing pattern remains valid *level by level* with no new code. Without FARGO the constraint is unnecessary and is not applied, which is why the `orbital_advection` toggle (§3.6) is part of Phase 0 rather than an unrelated convenience.

3.2 Where the policy is enforced

Enforcement is deliberately layered: a *veto* that prevents most violations from being requested, and a *fatal check* that catches anything that slips through.



(a) interior ring **allowed** (b) boundary annuli **allowed** (c) partial ring **fatal** (I2)

Figure 1: Refinement policy in the x–y plane. Red edges are the shear-periodic x_1 faces. (a) Interior ring: boundary at root level, refinement spans the full azimuth. (b) Both faces uniformly refined — allowed by I1 from Phase 0b onward, and the configuration that creates boundary octets in Phase 2b. (c) A patch that does not close in y violates I2 whenever orbital advection is on.

Site	Kind	Effect
`Mesh::BuildTreeFromScratch`	fatal	I1, I2 checked on the freshly built tree
`Mesh::BuildTreeFromRestart`	fatal	Same, on restart
`MeshRefinement::CheckForRefinement`	veto	Clears refine flags that would violate I1 (boundary blocks; graded cap)
`MeshRefinement::CheckForRefinement`	sync	Ring-wise OR/AND of flags to enforce I2 by construction
`MeshRefinement::AdaptiveMeshRefinement`	fatal	I1, I2 re-checked after `RedistAndRefineMeshBlocks`

`Mesh::CheckShearingBoxRefinement(ParameterInput*)` (`build_tree.cpp`) is the single implementation of both invariants. It returns immediately unless the mesh is multilevel and a `<shearing_box>` block exists. For I1 it scans `lloc_eachmb` for blocks with $l_{x1} = 0$ or $l_{x1} = N_{\mathrm{mb},x1}(\ell) - 1$ and requires $\ell_{\min} = \ell_{\max}$ over that set. For I2 it accumulates a `std::map` keyed on (ℓ, l_{x1}, l_{x3}) counting leaf blocks per ring and requires each count to equal the full azimuthal block count at that level. On a violation it prints the offending ring and dumps every refined block's logical location, which is what makes a failed AMR configuration diagnosable at all.

3.3 The graded refinement cap

Vetoing refinement of blocks that *sit* on the boundary is not enough: 2:1 balancing in `UpdateMeshBlockTree` propagates refinement outward from a newly refined block, and can reach the boundary columns from a few blocks away. Before commit `b5388c02` this turned an over-eager AMR criterion into a fatal abort mid-run.

The cap is a geometric room requirement. Let ℓ_b be the (uniform, by I1) boundary level and consider a block at level ℓ with $s = \ell - \ell_b \geq 1$ requesting refinement to $\ell + 1$. Balancing then requires a buffer column of every intermediate level between it and the boundary, of total width $(1 - 2^{-s})$ boundary-level columns on each side — and, expressed in integer logical coordinates *at the block's own level*,

$$2^{s+1} - 1 \;\leq\; l_{x1} \;\leq\; N_{\mathrm{mb},x1}(\ell) - 2^{s+1}. \tag{3}$$

The implementation reads exactly that:

```
if (lx1 == 0 || lx1 == (nmbx1-1)) {          // MB on boundary: never refine
  refine_flag.h_view(m+mbs) = 0;
} else if (lev >= (blev+1)) {                // graded cap on interior MBs
```

```
  std::int32_t gap = (1 << ((lev-blev)+1));
  if ((lx1 < (gap-1)) || (lx1 > (nmbx1-gap))) { refine_flag.h_view(m+mbs) = 0; }
}
```

With the cap in place, any I1 violation surviving to the post-update check is a genuine bug rather than a criterion asking for too much — which is what makes it reasonable to keep that check fatal.

3.4 Ring-wise flag synchronization

I2 is enforced *constructively* rather than by veto, immediately after the global flag `Allgatherv` in `CheckForRefinement`. Flags are collapsed into a map keyed on (ℓ, l_{x1}, l_{x3}) with the rule

$$\text{refine if } \textit{any} \text{ member is flagged;} \quad \text{derefine only if } \textit{every} \text{ member is flagged,} \tag{4}$$

and then written back to all members of the ring. The "any refine wins" direction guarantees that a criterion that fires anywhere in a ring produces a complete ring; the unanimity requirement for derefinement prevents a ring from being partially destroyed.

The ordering matters: this runs *after* the `Allgatherv`, on the globally consistent flag array, so every rank computes the identical map and the result is deterministic without any additional communication.

3.5 The derefinement-ordering fix

Ring-complete derefinement exposed a latent upstream bug. `UpdateMeshBlockTree` sorts the derefinement-candidate list finest-first so that `MeshBlockTree::Derefine`'s finer-neighbor veto operates on an already-updated tree, but used `&cllderef[ctnd-1]` as the end iterator — excluding the last element:

```
- std::sort(cllderef, &(cllderef[ctnd-1]), Mesh::GreaterLevel);
+ std::sort(cllderef, &(cllderef[ctnd]),   Mesh::GreaterLevel);
```

When a mixed-level derefinement left a fine-level parent in the unsorted last slot, coarser parents merging before it were vetoed *in an order-dependent way*: some parents of an annulus merged and some did not, splitting what should have been a ring-complete derefinement and tripping the I2 fatal. Outside the shearing box the bug only delayed some derefinements by one AMR interval, which is why it had gone unnoticed.

3.6 Non-FARGO (full-velocity) mode

`<shearing_box> orbital_advection` (default `true`) selects the frame. With `false`, `OrbitalAdvectionCC/FC` are simply not constructed (`porb_u = porb_b = nullptr`) while the shear-periodic BCs and the Coriolis/tidal source terms stay active, and the solver integrates the full shear flow directly. This mode exists for two reasons: it removes constraint I2 entirely (useful for arbitrary refinement patterns), and it provides an independent frame against which the FARGO path can be checked.

Consistency requires three coordinated changes.

(i) Source terms (`shearing_box_srcterms.cpp`). The shear-subtracted frame uses the reduced Coriolis coefficient $(2-q)\Omega_0$ on $\dot{m}_2$ because the background shear's contribution is carried analytically by orbital advection. In the full-velocity frame the velocities *include* the background, so the code applies the full Coriolis force plus the radial tidal force:

$$\dot{m}_1 \mathrel{+}= 2\Omega_0 m_2, \qquad \dot{m}_2 \mathrel{-}= 2\Omega_0 m_1, \qquad \dot{m}_1 \mathrel{+}= 2q\Omega_0^2 \rho\, x, \tag{5}$$

with the corresponding tidal work added to the total energy for an ideal EOS.

(ii) Boundary momentum/energy offsets (`shearing_box_cc.cpp`). The background azimuthal velocity jumps by $q\Omega_0 L_x$ across the box, so data wrapped through the shear-periodic boundaries must be offset to represent the *local* frame. After the unpack, for each ghost cell:

$$E \mathrel{+}= \delta v_y\, m_2 + \tfrac{1}{2}\rho\, \delta v_y^2, \qquad m_2 \mathrel{+}= \rho\, \delta v_y, \qquad \delta v_y = \begin{cases} +q\Omega_0 L_x & \text{at } ix_1 \\ -q\Omega_0 L_x & \text{at } ox_1 \end{cases} \tag{6}$$

Note the order: energy uses the pre-update m_2, so the momentum update must come second.

(iii) Initial conditions (`shwave.cpp`, `swing.cpp`). Every problem generator branch adds $v_y^{(0)} = -q\Omega_0 x$ to the initial azimuthal velocity when `orbital_advection = false`, with the matching kinetic-energy contribution.

MHD restriction. MHD orbital advection remaps face-centered B through EMFs; the correctness of that remap at fine/coarse interfaces (specifically, $\nabla\cdot B$ preservation) has not been analyzed, so MHD + FARGO + refinement remains fatal in `MHD::MHD`. Hydro is supported, and MHD without orbital advection is supported.

3.7 The shift-depth estimate and its runtime guard

Orbital advection communicates a buffer of depth `ng` + `maxjshift`, where `maxjshift` is estimated once at construction. The original estimate, $\lceil \mathrm{CFL}\cdot \max|x_1| \rceil$, omitted the $q\Omega_0$ factor. It was corrected to

$$\texttt{maxjshift} = \lfloor \mathrm{CFL}\,(q\Omega_0) \max|x_1| \rfloor + 1, \qquad \text{doubled on multilevel meshes,} \tag{7}$$

the doubling being headroom for fine blocks at large $|x_1|$, whose shift in their own (smaller) cell units is larger whenever Δt is not set by that same region.

Because that is still a heuristic, `OrbitalAdvectionCC::RecvAndUnpackCC` now *verifies* it every step against this step's Δt and each block's own geometry, before unpacking:

$$\max_m \left\lfloor \frac{q\Omega_0 \Delta t\, \max|x_1|_m}{\Delta x_{2,m}} \right\rfloor \;\le\; \texttt{maxjshift}, \tag{8}$$

aborting with a message that names the observed shift and the buffer depth. This converts a silent out-of-bounds read into a diagnosable failure.

3.8 Phase 0c: making the boundary state rebuildable under AMR

AMR renumbers MeshBlock GIDs and reassigns blocks to ranks on every update, but `x1bndry_mbgid`, the MPI request arrays, and the send/recv buffers were all built once in the `ShearingBox` constructor. Running AMR would have silently corrupted the shear exchange. The restructuring:

`ShearingBox::SetX1BndryMBs()`	List and request-array construction, extracted from the constructor and re-expressed against `pmy_pack` so it can run on the current pack. Grow-only `realloc` of `x1bndry_mbgid`; all requests reset to `MPI_REQUEST_NULL`.
`ShearingBox::AllocateBuffers()`	New pure virtual; `ShearingBoxCC` and `ShearingBoxFC` override it with the grow-only buffer reallocation extracted from their constructors. `ShearingBoxCC` now stores `nvar` because it needs it to size buffers outside the constructor.
`ShearingBox::ReinitAfterMeshUpdate()`	`SetX1BndryMBs()` followed by the virtual `AllocateBuffers()`.

The call site is `MeshRefinement::AdaptiveMeshRefinement`, placed after `RedistAndRefineMeshBlocks` and the post-update policy check, and *before* `InitBoundaryValuesAndPrimitives` — i.e. after the block metadata is rebuilt and before any boundary exchange touches the new mesh. It fires for `phydro->psbox_u`, `pmhd->psbox_u`, and `pmhd->psbox_b` when present.

The communicator is created once (`MPI_Comm_dup` in the constructor) and not re-duplicated; only the request arrays and buffers are rebuilt. Buffers grow but never shrink, which avoids reallocation churn when the block count oscillates; the extra slots are simply unused.

3.9 A user AMR criterion for testing: the moving ring

Exercising ring creation and destruction requires a criterion whose behavior is known in advance. `MovingRingRefine` (in `shwave.cpp`; an identical `SwingMovingRingRefine` in `swing.cpp`) refines every block whose x_1 extent overlaps

$$[x_c(t) - w,\ x_c(t) + w]\,, \qquad x_c(t) = x_{c0} + x_{\rm amp}\sin(\omega_r t), \tag{9}$$

and flags all others for derefinement. Because the criterion depends only on x_1 it naturally flags complete rings; the ring sync and boundary vetoes of §3.2 supply the rest of the policy. It runs on host data (block bounds only, no kernel) and is enrolled from the problem generator on both fresh starts and restarts, controlled by `<problem> amr_moving_ring` and the parameters `ring_xc0`, `ring_xamp`, `ring_freq`, `ring_hwidth`.

Two practical consequences of user-criterion AMR are worth recording, because they constrain what tests can prove: a run starts on the *unrefined* root mesh and refines within the first `refinement_interval` cycles, so bit-identity with an equivalent SMR run is structurally impossible; and `max_nmb_per_rank` must carry transient headroom for the moment when the old ring has not yet been destroyed and the new one already exists.

4 Phase 1: shear-periodic boundaries for the multigrid solver

4.1 The design decision

Equation (1) is an identification of the domain with its azimuthally displaced image. It changes *which cells are neighbors*, not *what the operator is*. So:

Design rule. The 7-point Laplacian, the smoother, the restriction and prolongation operators, and the coarse-grid solve are untouched. Shear-periodicity enters exclusively as a modified x_1 ghost fill — one that shifts the incoming plane by $\pm q\Omega_0 t_s L_x$ in y — applied identically at every level of the multigrid hierarchy.

Two properties of multigrid make the "identically at every level" clause cheap. First, the FAS V-cycle's fixed point satisfies the *finest*-level system; the coarse levels only accelerate convergence. Coarse-level remap quality therefore affects the convergence *rate*, not the answer. Second, because the shift is generally a non-integer number of cells at *every* level anyway, no level is a special case: one code path serves all of them.

4.2 The shear phase $q\Omega_0 t_s$

The box is strictly shear-periodic at times $t_n = nL_y/(q\Omega_0 L_x)$; measuring the offset from the most recent such instant keeps the shift — and hence the remap error — minimal:

```
Real MultigridDriver::ComputeShearQomt(Real time) const {
  Real tshear = ly/(mg_qshear_*mg_omega0_*lx);
  Real dts = time - std::floor(time/tshear)*tshear;
  return mg_qshear_*mg_omega0_*dts;
}
```

`MGGravityDriver::Solve` freezes `mg_qomt_` once per solve, from `pmesh->time`. This is deliberately the *same convention as the FFT solver*, so both solvers see identical boundary identifications at any instant and can be compared bit-for-bit. Note that hydro's own shear-periodic BC uses `time+dt` on the final RK stage; the resulting $O(\Delta t)$ offset between gravity and hydro is a pre-existing, FFT-validated convention and is intentionally *not* "fixed" in the multigrid path, since doing so would break the solver-to-solver comparison that validates it.

4.3 Data structures

Two members are added to `Multigrid` (`multigrid.hpp`):

`DvceArray4D<Real> *shear_plane_`	One global boundary plane per multigrid level, dimensioned `(2, nvar*ngh, gnz, gny)`. Index 0 selects the face: plane 0 is the *source for the inner ghosts* (gathered at the outer boundary), plane 1 the source for the outer ghosts.
`DvceArray2D<int> shear_goffs_`	Per-block global (y, z) cell offsets at the finest level, $(l_{x2} n_{x2},\ l_{x3} n_{x3})$, shifted right by the level index at use.
`int shear_reflev_`	Refinement offset of the x_1-boundary blocks above root (Phase 2b; zero on uniform grids and interior-only refinement).

and four on `MultigridDriver`: `mg_shear_enabled_`, `mg_qshear_`/`mg_omega0_`, `mg_qomt_` (frozen per solve), and `mg_remap_order_`.

Plane extents at level l (with $\texttt{ll} = \texttt{nlevel\_} - 1 - l$) are

$$\texttt{gny} = (N_{\mathrm{mb},x2} \ll \texttt{shear\_reflev\_})\,(n_{x2} \gg \texttt{ll})\,, \qquad \texttt{gnz} = (N_{\mathrm{mb},x3} \ll \texttt{shear\_reflev\_})\,(n_{x3} \gg \texttt{ll})\,, \tag{10}$$

i.e. the full y–z extent of the mesh at the boundary blocks' level, coarsened to this multigrid level.

4.4 The block-level fill: gather, remap, scatter

`Multigrid::FillShearGhostPack(qomt, order)` is three kernels submitted on one execution-space instance — stream-ordered, so no explicit fences are required between them.

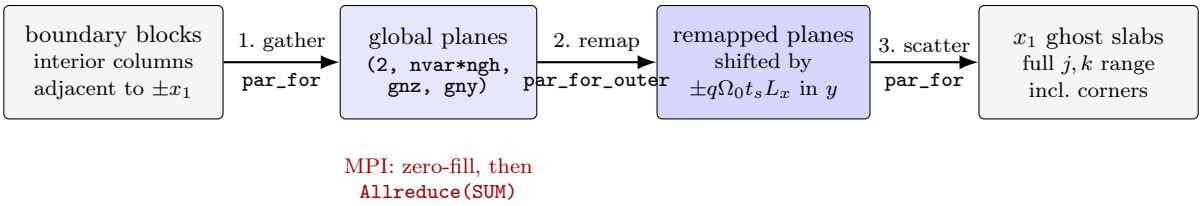


Figure 2: The block-level shear ghost fill. The only MPI seam is a single `Allreduce` between gather and remap.

1. Gather. Over all blocks and all interior (k, j), blocks whose `mb_bcs` carries `shear_periodic` on an x_1 face write their `ngh` interior columns adjacent to that face into the plane at global index $(\texttt{gk}, \texttt{gj})$ derived from `shear_goffs_`. Note the crossing: an *outer*-boundary block feeds plane 0, which will fill *inner* ghosts. Non-boundary blocks — including all interior fine blocks under Phase 2a — are skipped by the `mb_bcs` test, with no extra logic.

2. Remap. A team kernel over (face × var × depth, `gk`). For each row, the shift $y_{\rm sh} = \pm\, q\Omega_0 t_s L_x$ is split:

$$\texttt{joffset} = \lfloor y_{\rm sh}/\Delta y_l \rfloor\,, \qquad \epsilon = \mathrm{fmod}(y_{\rm sh}, \Delta y_l)/\Delta y_l. \tag{11}$$

The integer part is folded into a *periodic-wrapped scratch load* $q_{jf} = \mathrm{plane}[(jf - \texttt{PAD} - \texttt{joffset}) \bmod \texttt{gny}]$, and the fractional part is applied by the shared conservative kernels via Eq. (2). Because a full period is loaded before any write-back, the remap is safely in-place.

The scratch padding is `PAD = 3`: PPMX reads $j-2 \ldots j+2$ plus the face at $ju+1$, so three is exactly sufficient — and, crucially, independent of the shift size, because the integer part never enters the stencil. This is why $\texttt{ngh} = 1$ imposes no stencil limitation on the multigrid solver even though the shift can be many cells.

3. Scatter. Over the *full* slab $k, j \in [0, \texttt{ncells} + 2\texttt{ngh})$ — not just interior cells — with periodic wrap in y and z. Writing the slab's edge and corner ghosts matters: trilinear prolongation reads them. The read set (interior columns) and write set (ghost columns) are disjoint, so gather and scatter cannot race.

4.5 The root-grid fill

The root grid is a single replicated global array, so it needs no planes and is remapped in place by `MultigridDriver::RootShearBoundaryX1`, a single team kernel that loads the source column with the wrapped offset, applies the same remap fluxes, and writes the destination ghost column. It fills the interior-row ghosts only; the x_2/x_3 passes of `MGRootBoundary` that follow supply the ghost-corner rows by periodic wrap of these already-remapped rows. At $q\Omega_0 t_s = 0$ it reduces exactly to the plain periodic copy, because the remap fluxes vanish identically at $\epsilon = 0$ — which is what makes the "shear-periodic instant equals periodic, bitwise" test meaningful.

4.6 Hooks into the existing solver

Call site (file)	What it does
`MultigridDriver::MultigridDriver` multigrid_driver.cpp	`shear_periodic` is kept as its own `mg_mesh_bcs_` value instead of collapsing to `mg_zerofixed`
`MGGravityDriver::MGGravityDriver` mg_gravity.cpp	Enables shear, reads parameters, applies guards, calls `AllocateShearPlanes`
`MGGravityDriver::Solve` mg_gravity.cpp	Freezes `mg_qomt_` once per solve
`MultigridDriver::PhysicalBoundary` multigrid_tasks.cpp	Calls `FillShearGhostPack` after every boundary exchange
`MultigridDriver::MGRootBoundary` multigrid_driver.cpp	Calls `RootShearBoundaryX1` before the $x_1/x_2/x_3$ passes
`MultigridDriver::InitializeOctets` multigrid_driver.cpp	`shear_periodic` wraps like periodic in the neighbor search
`ApplyPhysicalBoundariesOctet` multigrid_driver.cpp	`shear_periodic` is a wrap, not a wall: no reflection applied

The ordering rule at every one of these sites is the same: *the shear fill runs after the ordinary (unsheared) exchange and overwrites its result.* The exchange performs the plain periodic copy through the neighbor tables; the shear fill replaces that copy with the y-remapped image over the whole slab. This is why the fill is placed at the top of `PhysicalBoundary`, which is the point every boundary exchange funnels through — including the final finest-level exchange that feeds `SelfGravity`'s ϕ ghosts.

4.7 Two corrections uncovered by the plumbing

The `mg_zerofixed` fall-through. Before this work, `shear_periodic` was not a recognized multigrid BC and fell through to `mg_zerofixed`. The consequence was severe and silent: the communication's (correct, unsheared) periodic copy was overwritten with a $\Phi = 0$ Dirichlet wall, and mean-source subtraction was disabled because the domain no longer looked fully periodic. Shearing-box multigrid gravity ran silently wrong. Recognizing `shear_periodic` throughout `mg_mesh_bcs_` fixes both.

Mean-source subtraction. A fully periodic Poisson problem is solvable only if the source has zero mean. `shear_periodic` is a periodic identification, not a physical boundary, so it must count toward "fully wrapped":

```
bool wrap_all = true;
for (int f = 0; f < 6; ++f) {
  BoundaryFlag mbc = pmy_mesh_->mesh_bcs[f];
  wrap_all = wrap_all && (mbc == BoundaryFlag::periodic ||
                          mbc == BoundaryFlag::shear_periodic);
}
if (!wrap_all) { fsubtract_average_ = false; }
```

4.8 Driver-independent task-list execution

Multigrid task lists were run through `Driver::ExecuteTaskList`, which dereferences the `Driver*`. Problem generators are constructed *before* the `Driver` exists, so a static Poisson solve from a pgen (`pgrav->Solve(nullptr, 1)`) crashed with `solver = multigrid`. `MultigridDriver::ExecuteMGTaskList` replicates the runner but tolerates `pdriver == nullptr`,

which is safe because every multigrid task ignores its `Driver*` argument. Together with a null guard in the non-convergence branch of `SolveIterative`, this makes static solves through the full multigrid path a supported configuration — the mechanism behind the static Poisson tests.

4.9 The MPI seam

Multi-rank support required exactly one collective, at one place (Fig. 2). Each rank gathers only its own boundary blocks' columns into a zero-filled plane; one `MPI_Allreduce(MPI_IN_PLACE, ..., MPI_SUM)` then reconstructs the global plane on every rank; remap and scatter proceed rank-locally (redundantly, but the planes are small).

Two properties make this exact rather than merely adequate. First, every plane cell has *exactly one* writer — boundary blocks tile the level's y–z extent — so the summed contributions are one value plus zeros, and the result is independent of reduction order: the multi-rank solve is bit-identical to the serial one. Second, all ranks reach the collective in lockstep, because every early-return at the call site (`shear_plane_ == nullptr`, `ncells < 1`, `mg_qomt_ == 0`) is a level-global condition, not a rank-local one. The `Kokkos::fence()` before the call is required because the gather kernel is asynchronous.

The root grid needs no communication (it is replicated), and the octet fill of Phase 2b needs none either (octets are replicated). The seam is compiled out entirely in serial builds.

The same commit made the `fft_poisson` pgen's diagnostics MPI-aware: the density mean and all residual/error norms were rank-local reductions written under the FFT solver's single-rank assumption, which corrupted the printed RHS mean and scaled the L_2 norms by $1/\sqrt{N_{\mathrm{ranks}}}$ under MPI.

4.10 Phase-1 guards and parameters

Condition	Reason
x_2 or x_3 not periodic	The shear identification assumes a periodic azimuth and vertical; the plane wrap would be wrong
`<gravity> mg_bc` set	An explicit MG boundary override cannot be combined with the shear identification
`root_on_host = true`	The root shear fill is a device kernel
`mg_nghost > 1`	Untested; also the octet fill of Phase 2b assumes `ngh` $= 1$ (octet interiors are 2 cells)
`mg_remap` unrecognized	Must be `dc`, `plm` (default), or `ppmx`

5 Phase 2: multigrid gravity on refined shearing-box meshes

Phase 2 is decomposed into three steps chosen so that each one isolates a single new mechanism, and each can be validated against the previous step as a regression.

Step	Mesh configuration	New mechanism
2a	Interior rings; boundary at root level	none (guard relaxation)
2b	Boundary annuli; both faces refined uniformly	sheared *octet* ghosts
2c	Adaptive refinement	rebuild-on-remesh

5.1 Phase 2a: interior rings

Phase 2a required *no changes to the fill at all*, and understanding why is the point of the step.

The shear fill iterates over blocks and gates on `mb_bcs`. Interior fine blocks do not carry `shear_periodic` on any x_1 face, so the gather and scatter skip them automatically. Every *participating* block is at the root level, so the plane geometry — sized in root-block units at construction — remains correct. The refined interior is handled entirely by the pre-existing octet machinery, which is level-agnostic and does not touch the shear faces.

What changed was the guard. The Phase-1 blanket `multilevel` fatal became a scan requiring every block on an x_1 face to be at the root level, with AMR still fatal (deferred to 2c) and refined boundary faces still fatal (deferred to 2b).

5.2 Phase 2b: sheared octet ghosts

Boundary annuli — both x_1 faces uniformly refined over their full x_2/x_3 extent, allowed by I1 since Phase 0b — put multigrid *octets* on the shear-periodic faces. Those octets exchange x_1 ghosts through the octet neighbor table, which wraps periodically (`InitializeOctets`, patched in Phase 1) but *unsheared*. They need the same treatment as the blocks.

Why this is a separate implementation. The octet regime differs from the block regime in every respect that matters to the fill: octet buffers live in *host* memory, are *replicated on every rank*, and have a fixed 2^3 interior. So `FillShearOctetGhosts` is the Phase-1 algorithm — gather, remap, scatter, same sign convention — re-expressed as serial host code over a `std::vector` plane, with no MPI seam at all.

The scalar remap twins. The conservative remap kernels are Kokkos *team* kernels and cannot be called from serial host code. Rather than duplicate the reconstruction logic, `remap_fluxes.hpp` gains scalar per-face twins — `DC_RemapFlxFace`, `PLM_RemapFlxFace`, `PPMX_RemapFlxFace` — each returning the remap flux at one face of a padded row, with the same convention as the team versions (faces $jf \in [\texttt{PAD}, \texttt{PAD}+\texttt{gny}]$). The upwind cell is $c = jf - 1$ for $\epsilon > 0$ and $c = jf$ for $\epsilon < 0$, matching which face the team kernels write. The device path is untouched, which is what preserves the Phase-1 and Phase-2a results bit-for-bit.

The header carries an explicit maintenance rule: *the math must stay in lockstep with the team versions; any change to one is a change to both.*

Gating. `oct_shear_face_` is a per-octet-level flag set in `InitializeOctets`: level l is flagged when its octets reach *both* $l_{x1} = 0$ and $l_{x1} = (\texttt{nrbx1\_} \ll l) - 1$. Under I1 either both faces are tiled at a level or neither is, so a single boolean per level is sufficient.

Hook points. Every consumer of octet ghosts funnels through two chokepoints, and the fill is placed at both, after the per-octet exchange:

- `SetBoundariesOctets(fprolong, folddata)` — per level, during the cycle: `FillShearOctetGhosts(lev, folddata)`;
- `SetOctetBoundariesBeforeTransfer(folddata)` — all levels, before the root transfer, since these ghosts feed `SetFromRootGrid` for the boundary blocks' coarsest levels.

The `folddata` argument propagates to the fill: when set, `Uold` is gathered, remapped, and scattered alongside `U` in the same pass (the plane carries a leading `nfield` axis of size 1 or 2).

Plane sizing at the boundary level. With boundary annuli the participating blocks sit at a uniform level *above* root, so the block planes must span the y–z extent at *that* level. `AllocateShearPlanes` determines it by scanning for the first block on an x_1 face — under I1, any face block gives the answer — and stores `shear_reflev_` $= \ell - \ell_{\mathrm{root}}$, which then multiplies `gny` and `gnz`.

Guard relaxation. The 2a root-level requirement weakens to exactly the hydro invariant I1: the x_1-face blocks must share one uniform level, reported with the observed level span on violation. Interior rings (boundary at root) and boundary annuli (boundary above root) both pass through the same check.

A note on coarse-level fidelity. As in Phase 1, the V-cycle's fixed point is set by the finest block level, so octet-level ghost quality affects only the convergence rate. The fill is still done consistently at every octet level, because an inconsistent coarse-grid correction is what would degrade the sheared contraction factor away from its periodic value.

5.3 Phase 2c: adaptive refinement

AMR invalidates every shear structure that was sized at construction: plane extents (if the boundary level changes), the per-block offset table (load balancing can reassign blocks without changing their count), and the octet face flags.

The fix rides machinery that already exists. `MultigridDriver::PrepareForAMR` is called at the top of every `Solve` and already detects any remesh via an update counter, rebuilding the multigrid arrays and ϕ. Two changes complete the picture:

1. `Multigrid::AllocateShearPlanes` is made *idempotent*: the plane array is allocated only if null, `Kokkos::realloc` resizes each plane if the boundary level changed, `shear_reflev_` is re-derived, and the offset table is recomputed from the current logical locations. It is then called from the same rebuild branch of `PrepareForAMR`.
2. The octet face flags need no new code: `InitializeOctets` is already part of the rebuild and reassigns `oct_shear_face_` from the new octet set.

The constructor's `adaptive` fatal is removed. The invariant it was protecting is not abandoned but relocated: `Mesh::CheckShearingBoxRefinement` re-enforces I1 and I2 after *every* AMR update (§3.2), so by the time `PrepareForAMR` runs, the mesh is known to satisfy the uniform-boundary-level condition that the plane sizing assumes.

Finally, the swing problem generator gains the moving-ring criterion of §3.9, so that AMR + shear + multigrid gravity can be exercised on a problem with a known analytic reference.

5.4 End-to-end control flow

6 Rank-consistent integer shifts

This fix is not part of any phase but is required for all of them at scale, and is recorded here because the failure mode is instructive.

Symptom. A 256^2 run aborted at 64 ranks with Cray MPICH *message truncated*, deterministically at cycle 7900, $t = 13.98274$, as $q\Omega_0 t_s$ approached the shear-periodic wrap — with a 3:4:5 size signature characteristic of a disagreement about how many blocks a message spans.

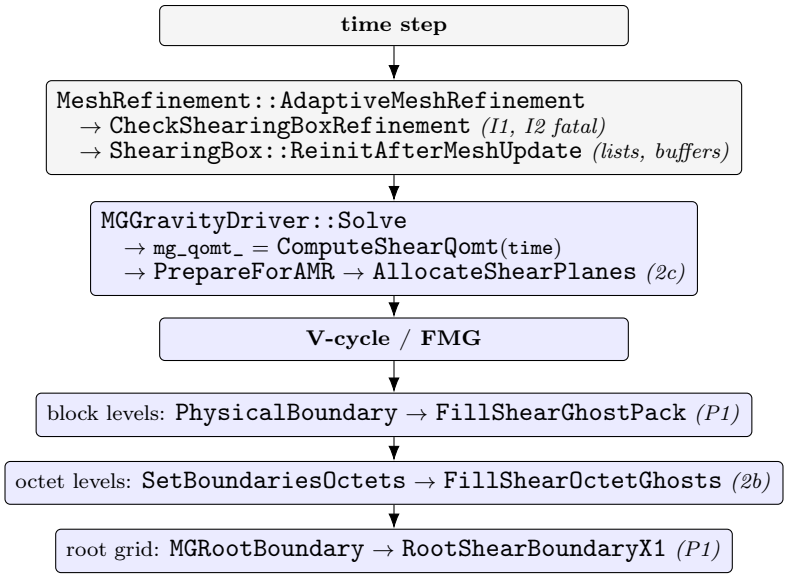


Figure 3: Where each piece runs. Grey: mesh/hydro side (Phase 0). Blue: multigrid solver (Phases 1–2).

Cause. The integer cell shift of the x_1-boundary exchange was computed independently on the sending and receiving blocks as

$$\texttt{joffset} = \lfloor y_{\rm shear}/\texttt{mb\_size.dx2} \rfloor \,. \tag{12}$$

Per-block `dx2` values carry last-ulp roundoff — they are derived from each block's own extents — so whenever $y_{\rm shear}/\Delta x_2$ sits within an ulp of an integer, the two sides can differ by one cell. That flips the case-1/2/3 selection in the exchange, and hence the `joffset`-dependent MPI *message sizes*, desynchronizing sender and receiver.

Fix. `ShearingBox::ConsistentDx2(gid)` derives Δx_2 from global quantities only, with the exact power-of-two level factor:

$$\Delta x_2(\ell) = \frac{x_{2,\rm max} - x_{2,\rm min}}{\left(N^{\rm root}_{{\rm mb},x2}\, n_{x2}\right) \ll (\ell - \ell_{\rm root})}. \tag{13}$$

Every input is bitwise identical on every rank, so every block of a level — and both endpoints of every exchange — agrees exactly.

Completeness. The fix required all *seven* `joffset` sites, found in two passes. The first (`89904070`) covered the sender and unpack sides in `shearing_box_cc.cpp` and `shearing_box_fc.cpp`. The second (`ca63f727`) covered `shearing_box_tasks.cpp`, which carries its *own* copy of the case-1/2/3 logic to post the `MPI_Irecvs` (`InitRecv`, `ClearRecv`, `ClearSend`) with its own per-block `dx2`. That the sender/recv-poster disagreement is deterministic for a given mesh and $y_{\rm shear}$ is precisely why the crash reproduced at the identical cycle in both attempts.

A sweep of the tree confirmed no other sites: FARGO's `joffset` is sender-local (pack-only, fixed-size buffers behind the I1 guard) and the FFT and multigrid solver shifts already derive from global quantities. Remap ϵ values shape content only, never message sizes, so last-ulp differences there are harmless.

7 Input-file interface

7.1 Parameters

Block	Parameter	Default	Meaning
`<shearing_box>`	`qshear`	—	q
	`omega0`	—	Ω_0
	`orbital_advection`	`true`	`false` selects the full-velocity (non-FARGO) frame and lifts I2
`<gravity>`	`solver`	—	`multigrid` or `fft`
	`four_pi_G`	—	$4\pi G$
	`mg_remap`	`plm`	Remap order for the shear ghost fill: `dc`, `plm`, `ppmx`
	`mg_nghost`	1	Must be 1 with shear
	`root_on_host`	`false`	Must be false with shear
	`threshold`	—	< 0 selects fixed-iteration mode
	`niteration`	—	Cycle count in fixed mode
`<problem>`	`amr_moving_ring`	`false`	Enroll the prescribed moving-ring criterion
	`ring_xc0`, `ring_xamp`	0	Ring center and oscillation amplitude
	`ring_freq`	1	Ring oscillation frequency
	`ring_hwidth`	—	Ring half-width
`<mesh>`	`ix1_bc`, `ox1_bc`	—	`shear_periodic` (both, or neither)
`<mesh_refinement>`	`refinement`	—	`static` or `adaptive`
	`max_nmb_per_rank`	—	Needs transient headroom under moving-ring AMR

7.2 Reference inputs

File (under `inputs/`)	Configuration
Phase 0 — hydro only	
`shearing_box/hydro_incompress_shwave_smr.athinput`	SMR, no FARGO
`shearing_box/hydro_incompress_shwave_smr_ring.athinput`	Annular ring, FARGO
`shearing_box/hydro_incompress_shwave_smr_ring2.athinput`	Two nested rings
`shearing_box/hydro_incompress_shwave_smr_bndry{,2}.athinput`	Refined boundary faces (I1)
`shearing_box/hydro_incompress_shwave_amr_ring.athinput`	Moving ring, AMR
`shearing_box/epicycle_smr_bndry.athinput`	Epicycle, refined boundary
`shearing_box/epicycle_amr_ring{,2,_nofargo}.athinput`	Epicycle under AMR
Phases 1–2 — with gravity	
`tests/fft_poisson_shear.athinput`	Static Poisson, uniform (FFT/MG A/B)
`tests/mg_poisson_shear_smr.athinput`	Static Poisson, interior ring (2a)
`tests/mg_poisson_shear_bndry.athinput`	Static Poisson, boundary annuli (2b)
`tests/swing_selfgrav.athinput`	Swing amplification, uniform
`tests/swing_selfgrav_smr.athinput`	Swing, interior ring (2a)
`tests/swing_selfgrav_bndry.athinput`	Swing, boundary annuli (2b)
`tests/swing_selfgrav_amr.athinput`	Swing, moving ring, AMR (2c)
`tests/ts41_sin3.athinput`, `tests/ts43_jeans.athinput`	Tomida & Stone (2023) §4 consistency

Because `solver` is a single input key, most gravity inputs are a one-override A/B against the FFT oracle: adding `-gravity/solver=fft` on the command line runs the identical problem through the independent solver.

7.3 Building

```
cmake -B build -D Athena_ENABLE_MPI=ON     # add -D Kokkos_ENABLE_CUDA=ON for GPU
cmake --build build -j
```

`multigrid/multigrid_shear.cpp` is registered in `src/CMakeLists.txt`; the MPI seam of §4.9 compiles out entirely in serial builds, so serial and MPI builds produce identical output on identical problems.

8 Guard reference

Every fatal condition introduced by this work, with the mechanism it protects.

Location	Condition	Protects
`Mesh::CheckShearingBoxRefinement`	x_1-face blocks span more than one level	I1: same-level wrap and same-size blocks along each face
`Mesh::CheckShearingBoxRefinement`	Incomplete azimuthal ring, FARGO on	I2: same-level, same-size x_2-face neighbors for the per-level shift
`OrbitalAdvectionCC::RecvAndUnpackCC`	Actual shift > `maxjshift`	Buffer depth; converts an out-of-bounds read into a diagnosable abort

Location	Condition	Protects
`MHD::MHD`	FARGO + refinement	Face-centered B remap at fine/coarse interfaces ($\nabla\cdot B$) is unanalyzed
`MGGravityDriver`	x_2 or x_3 not periodic	Plane wrap assumes periodic azimuth/vertical
`MGGravityDriver`	`mg_bc` set with shear	Conflicting boundary specification
`MGGravityDriver`	`root_on_host` with shear	Root fill is a device kernel
`MGGravityDriver`	`mg_nghost > 1` with shear	Untested; octet fill assumes one ghost
`MGGravityDriver`	Unrecognized `mg_remap`	Typo protection
`MGGravityDriver`	x_1-face blocks span more than one level	Plane sizing (`shear_reflev_`) and octet face tiling

9 Known limitations and deferred work

1. **Mixed-level shear boundaries.** I1 forbids them. Supporting them means combining the y-shift with prolongation *inside* the shear communication, for both the hydro exchange and the multigrid fill.

2. **MHD orbital advection with refinement.** Fatal. The face-centered B remap at fine/coarse interfaces needs a $\nabla\cdot B$ analysis before the guard can be lifted. MHD without orbital advection, and hydro with it, are both supported.

3. **`root_on_host` with shear.** Fatal. `RootShearBoundaryX1` is a device kernel; a host twin (analogous to the scalar remap twins of §5.2) would lift this.

4. **`mg_nghost > 1` with shear.** Fatal. The block fill is written for general `ngh`, but `FillShearOctetGhosts` assumes `ngh` $= 1$ because octet interiors are two cells deep.

5. **Redundant remap work under MPI.** Every rank remaps the whole plane after the `Allreduce`. This is what makes multi-rank results bit-identical to serial ones, and the planes are small relative to the volume data, but it is redundant work that a partitioned remap would avoid — at the cost of that guarantee.

6. **The stopping rule on refined meshes.** Stagnation-based termination (`threshold = 0`) can hit the cycle cap mid-run on the wound-up $k_x \sim 0$ mode, whose coarse-grid correction crosses the sheared x_1 wrap. Refined shearing-box inputs therefore default to FMG with a fixed iteration count. This is a practical convergence-rate issue, not a correctness one — the fixed point is unaffected — but it is the one place where the sheared identification visibly degrades multigrid efficiency.

7. **The $O(\Delta t)$ phase offset.** Gravity freezes the shear phase at `pmesh->time`; hydro's shear-periodic BC uses `time+dt` on the final RK stage. This is a pre-existing convention retained deliberately so the multigrid and FFT solvers remain directly comparable; revisiting it means revisiting both solvers and their validation together.